Reviewer Pack — Positivstellensatz Rank Lower-Bounds SOS Max-CUT Degree

Entry 8f0ca0b1270f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 07:43:20 UTC

Positivstellensatz Rank Lower-Bounds SOS Max-CUT Degree

Entry ID: `8f0ca0b1270f` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-08 07:43:20 UTC

1. Conjecture

Field A (mathematical branch): Real Algebraic Geometry **Field B** (complexity object): Sum-of-Squares Degree

Statement:

> For a max-CUT instance with n variables, if its corresponding polynomial $f \in \mathbb{R}[x_1,\ldots,x_n]$ is non-negative but not a sum of squares, then the SOS degree required to approximate f within $1-\varepsilon$ is $\Omega(\log n)$.

Rationale (proposer's reasoning):

> Positivstellensatz characterizes non-negativity via algebraic certificates, while SOS hierarchies approximate this via bounded-degree sums of squares. Non-SOS-non-negative polynomials force higher-degree approximations, revealing structural gaps between algebraic geometry and convex optimization.

Taxonomy category: `NISAN_WIGDERSON_DESIGNS` (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `f43897a91863e7dd`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

Barrier	Verdict	Confidence	LLM₁	LLM₂

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def isqrt(n):
        x = n
        y = (x + 1) // 2
        while y < x:
            x, y = y, (y + x) // 2
        return x

    def matrix_multiply(A, B):
        m, k = len(A), len(B[0])
        result = [[0] * k for _ in range(m)]
        for i in range(m):
            for j in range(k):
                for l in range(len(B)):
                    result[i][j] += A[i][l] * B[l][j]
        return result

    def gaussian_elimination(A, b):
        n = len(b)
        for i in range(n):
            max_row = i
            for j in range(i + 1, n):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            b[i], b[max_row] = b[max_row], b[i]
            for j in range(i + 1, n):
                factor = A[j][i] / A[i][i]
                A[j][i:] = [A[j][k] - factor * A[i][k] for k in range(i, n)]
                b[j] -= factor * b[i]
        x = [0] * n
        for i in range(n - 1, -1, -1):
            x[i] = (b[i] - sum(A[i][j] * x[j] for j in range(i + 1, n))) / A[i][i]
        return x

    def is_positive_definite(M):
```

```
    n = len(M)
    for i in range(n):
        if M[i][i] <= 0:
            return False
        for j in range(i + 1, n):
            factor = M[j][i] / M[i][i]
            M[j][i:] = [M[j][k] - factor * M[i][k] for k in range(i, n)]
    return True

def is_sum_of_squares(M):
    n = len(M)
    if not is_positive_definite(M):
        return False
    x = gaussian_elimination(M, [0] * n)
    return all(x[i]**2 <= M[i][i] for i in range(n))

def max_cut_instance(n):
    edges = [(i, j) for i in range(n) for j in range(i + 1, n)]
    random.shuffle(edges)
    return [random.choice([0, 1]) for _ in edges]

def construct_polynomial(instance):
    n = len(instance)
    poly = sum(1 - instance[i] * instance[j] for i, j in enumerate(instance))
    return poly

def moment_matrix(poly, n):
    m = len(poly)
    M = [[0] * (m + 1) for _ in range(m + 1)]
    for i in range(m):
        for j in range(i, m + 1):
            if j == i:
                M[i][j] = sum(1 for x in poly if x == i)
            else:
                M[i][j] = sum(x * y for x, y in zip(poly[:i], poly[j - i:]))
    return M

def sos_degree(M, epsilon):
    n = len(M)
    d = 0
    while True:
        d += 1
        A = [[0] * (d + 1) for _ in range(d + 1)]
        b = [0] * (d + 1)
        for i in range(n):
            for j in range(i, n):
                if M[i][j] != 0:
                    A[i % (d + 1)][j % (d + 1)] += M[i][j]
                    b[i % (d + 1)] += M[i][j]
        x = gaussian_elimination(A, b)
        if all(x[i]**2 <= M[i][i] for i in range(n)):
            return d

n = 40
instance = max_cut_instance(n)
poly = construct_polynomial(instance)
```

```
    M = moment_matrix(poly, n)

    if not is_positive_definite(M):
        return {
            "metric_name": "SOS Degree",
            "metric_value": None,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "not_positive_definite"
        }

    if is_sum_of_squares(M):
        return {
            "metric_name": "SOS Degree",
            "metric_value": None,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "sum_of_squares"
        }

    epsilon = 0.95
    d = sos_degree(M, epsilon)

    return {
        "metric_name": "SOS Degree",
        "metric_value": d,
        "instances_tested": 1,
        "conjecture_holds": d >= math.log(n),
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2**i + 17 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(re
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results if r["metric_va
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample='' first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_992de759.py", line 149, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_992de759.py", line 112, in run_trial
    M = moment_matrix(poly, n)
        ^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_992de759.py", line 83, in moment_matrix
    m = len(poly)
        ^^^^^^^^^
TypeError: object of type 'int' has no len()
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed due to TypeError; 'poly' was an integer instead of a polynomial object | next: Debug moment_matrix() implementation to ensure 'poly' is a sequence-type object

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	43026
2	preregistration	ollama_remote	qwen3:8b	0	0	20014
3	novelty	ollama_remote	qwen3:8b	0	0	16572
4	novelty	ollama_remote	qwen3:8b	0	0	9946
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16515
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16243
7	judge	ollama_remote	qwen3:8b	0	0	9689

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 132004 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/8f0ca0b1270f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/8f0ca0b1270f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/8f0ca0b1270f.tar.gz` (if generated)